Hochschule für Technik
und Wirtschaft Berlin

University of Applied Sciences

Live-Farbsegmentierung im Kamerabild

Laborprotokoll

Name des Studienganges

Computerengineering

Fachbereich 1

vorgelegt von

Paul Seelig

Datum:

Berlin, 13.04.2026

Inhaltsverzeichnis

1. Ziel 2
2. Ablauf 2
3. Zusammenfassung 5
4. Literatur 5

1. Ziel

Das Ziel der Übung war es die Aufnahme einer Webcam oder Kamera, während der Laufzeit des Programmes in den HSV-Farbraum zu konvertieren und diesen ggf. per Schieberegler zu kontrolieren. Hierfür wird die Bildverarbeitungsbibliothek Opencv verwendet, die Python implementation. [1]

2. Ablauf

Zuerst soll die Webcam vom Programm erkannt und weiterverarbeitet werden. Hierzu wird in der Funktion `cv2.VideoCapture(0)` die Webcam erkannt und in die Variable `cap` gelesen.

Danach wird in einer Schleife die Kamera ausgelesen `cap.read()` und in die Variablen `ret, img` gelesen.

Um aus der Schleife auszutreten muss der Knopf `q` für Quit gedrückt werden. Das führt dazu das die Fenster sich schließen und die Kamera nicht mehr aktiv sendet.

```
cap = cv2.VideoCapture(0)
...
  while(1):
    ret, img = cap.read()
    ...

    if cv2.waitKey(1) & 0xFF == ord("q"):
      break

cap.release()
cv2.destroyAllWindows()
```

- Konvertiere jedes Frame in den HSV-Farbraum (cv2.cvtColor). [2]

```
hsv = cv2.cvtColor(img, cv2.COLOR_RGB2HSV)
```

- Nutze cv2.createTrackbar, um die HSV-Schwellwerte zur Laufzeit per Schieberegler einzustellen. So kannst du interaktiv die passenden Werte für verschiedene Farben finden.

```
# Create windows
cv2.namedWindow('HSV')
cv2.namedWindow('Regler')

# Create trackbars ONCE (outside the loop)
# Taken from my previously submitted python file
cv2.createTrackbar('Hue_lower', 'Regler', 0, 179, lambda x: None)
cv2.createTrackbar('Saturation_lower', 'Regler', 0, 255, lambda x: None)
cv2.createTrackbar('Value_lower', 'Regler', 0, 255, lambda x: None)

cv2.createTrackbar('Hue_upper', 'Regler', 170, 179, lambda x: None)
cv2.createTrackbar('Saturation_upper', 'Regler', 255, 255, lambda x: None)
cv2.createTrackbar('Value_upper', 'Regler', 255, 255, lambda x: None)

...
```

```
while(1):
  ...
  Hue_L = cv2.getTrackbarPos('Hue_lower', 'Regler')
  Saturation_L = cv2.getTrackbarPos('Saturation_lower', 'Regler')
  Value_L = cv2.getTrackbarPos('Value_lower', 'Regler')

  Hue_U = cv2.getTrackbarPos('Hue_upper', 'Regler')
  Saturation_U = cv2.getTrackbarPos('Saturation_upper', 'Regler')
  Value_U = cv2.getTrackbarPos('Value_upper', 'Regler')
```

- Erzeuge mit cv2.inRange eine binäre Maske, die nur Pixel im gewählten Farbbereich enthält.

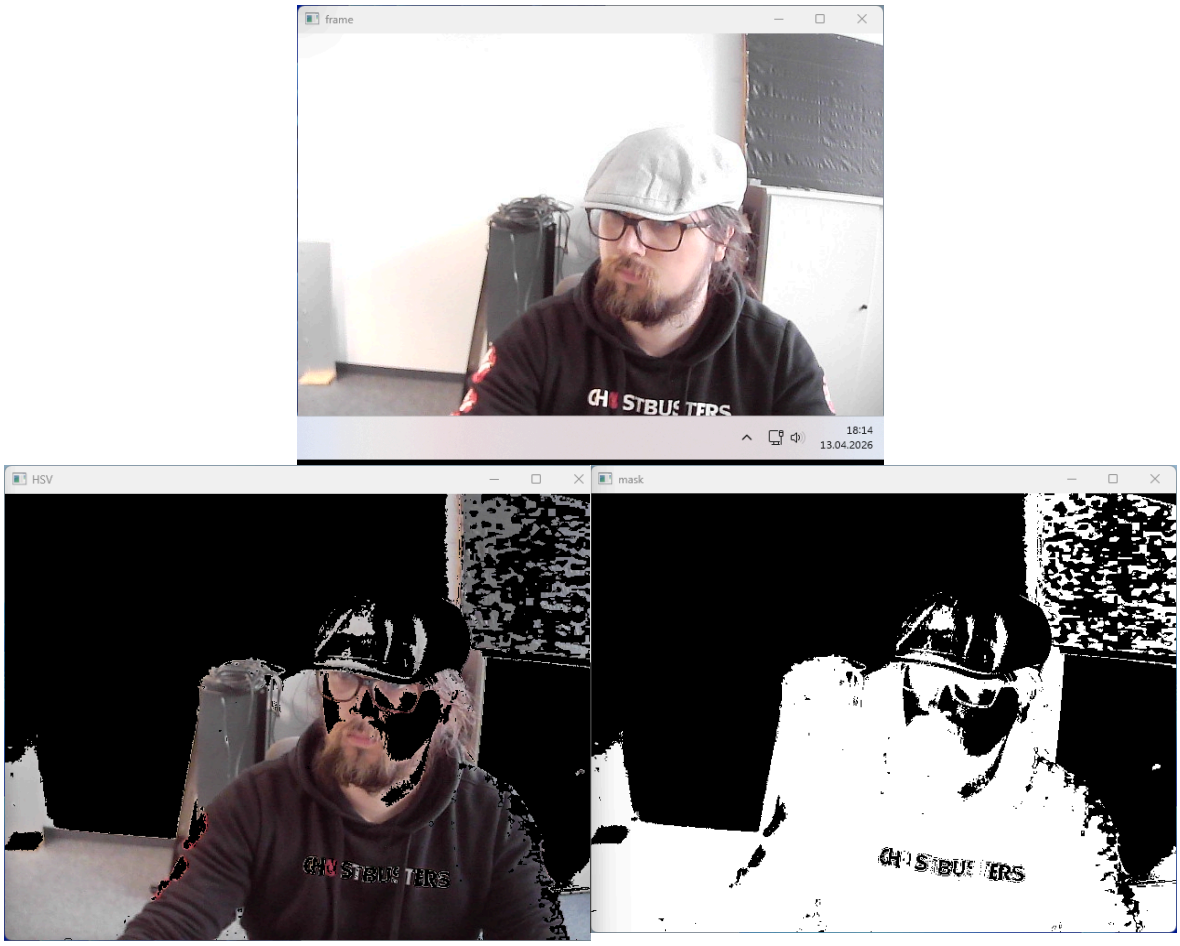
```
mask = cv2.inRange(hsv, lower, upper)
```

- Wende die Maske auf das Originalbild an (cv2.bitwise_and), sodass nur die segmentierten Bereiche sichtbar sind.

```
res = cv2.bitwise_and(img,img, mask= mask)
```

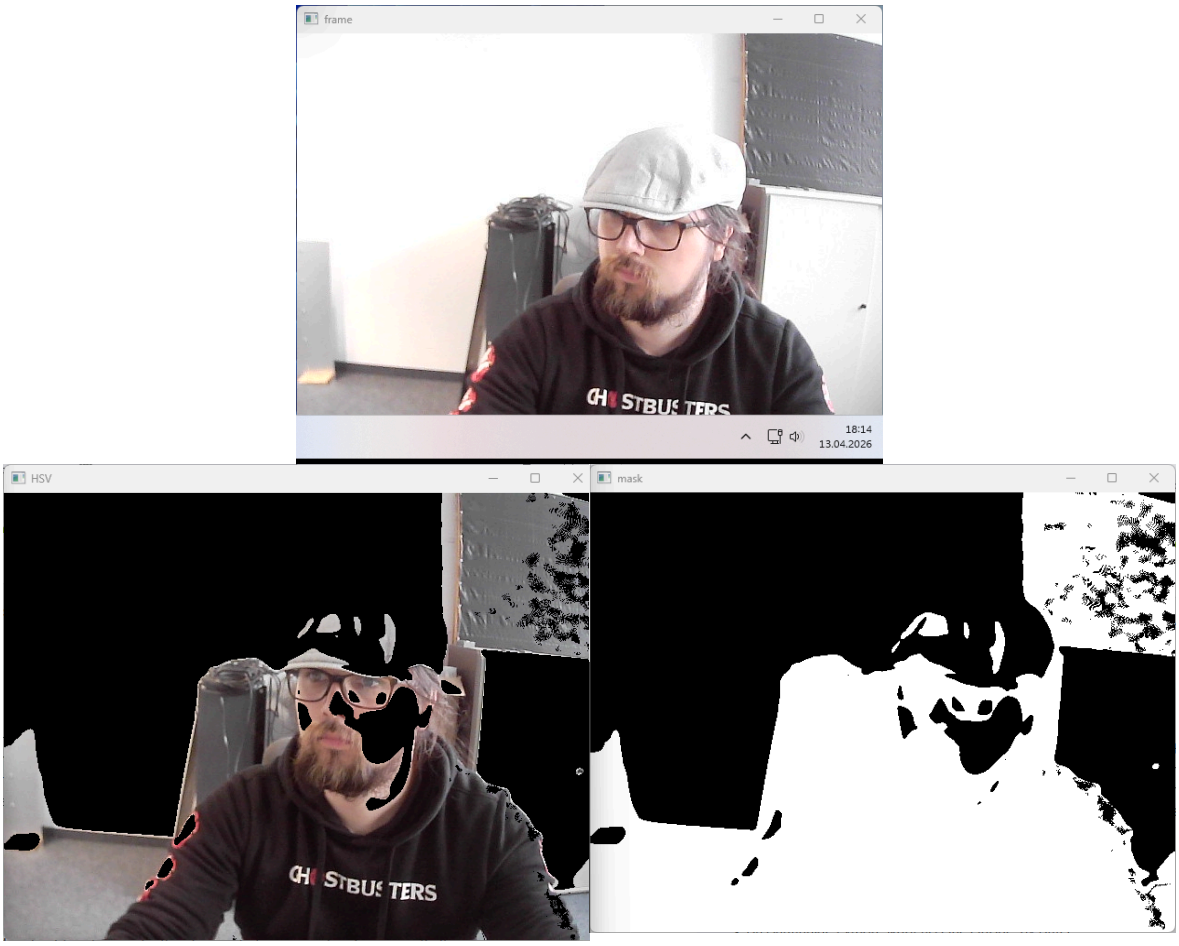
- Zeige das Originalbild, die Maske und das gefilterte Ergebnis nebeneinander an.

```
cv2.imshow('frame',img)
cv2.imshow('mask',mask)
cv2.imshow('HSV',res)
```



- Ein Gaußfilter (cv2.GaussianBlur) vor der Farbsegmentierung kann helfen, Rauschen zu reduzieren

```
blur = cv2.GaussianBlur(img, (15, 15), 0)
hsv = cv2.cvtColor(blur, cv2.COLOR_RGB2HSV)
```



3. Zusammenfassung

4. Literatur

[1] G. Bradski, „The OpenCV Library“, *Dr. Dobb's Journal of Software Tools*, 2000.

[2] OpenCV, „Changing Colorspaces“. Zugegriffen: 14. April 2026. [Online]. Verfügbar unter: https://docs.opencv.org/4.x/df/d9d/tutorial_py_colorspaces.html